

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

Aim: To design and implement FSM-based controllers for sequential digital systems.

Software required:

Vivado Simulator

Theory:

A Finite State Machine (FSM) is a sequential digital system that operates through a finite number of predefined states. The state of the system changes according to the current state and the applied input. FSMs are commonly used to design digital controllers such as sequence detectors, traffic-light controllers, communication controllers, and control units.

An FSM mainly consists of:

- Present State: The current state of the system.
- Input: Determines the transition from one state to another.
- Next State Logic: Determines the next state based on the present state and input.
- State Register: Stores the current state.
- Output Logic: Generates the required output.

There are two commonly used types of FSMs:

Mealy FSM

In a Mealy machine, the output depends on both the present state and the current input.

$$Output = f(P, present\ State, Input)$$

Therefore, the output can change immediately when the input changes.

Moore FSM

In a Moore machine, the output depends only on the present state.

$$Output = f(Present\ State)$$

The output generally changes when the state changes.

For Verilog implementation, FSMs are commonly described using:

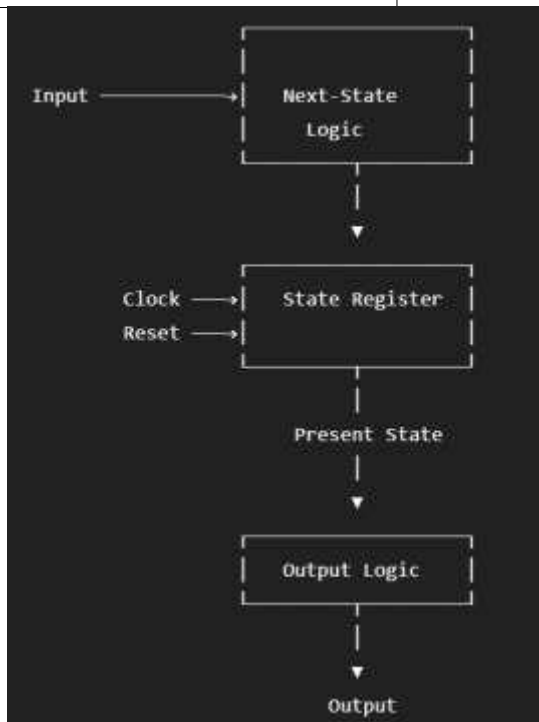
1. State declaration
2. State register
3. Next-state logic
4. Output logic

A reset state is used to initialize the FSM to a known state. This ensures predictable operation when simulation begins.

The FSM can be verified using a simulation waveform by applying different input sequences and observing the corresponding state transitions and outputs.

Block diagram:

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025



The FSM receives an input and determines the next state using the next-state logic. The state register stores the current state according to the clock. The output logic generates the required output based on the FSM state and, depending on the FSM type, the input.

Truth Table:

Present State	Input = 0	Input = 1
S0	S0	S1
S1	S2	S1
S2	S0	S1

Where:

- S0: No matching sequence
- S1: 1 detected
- S2: 10 detected
- When 1 is received in S2, the sequence 101 is detected.

For a Mealy sequence detector, the output is:

Present State	Input	Next State	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S1	1

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

The output becomes 1 when the sequence 101 is detected.

Boolean Expression:

For the selected FSM, the output is asserted when:

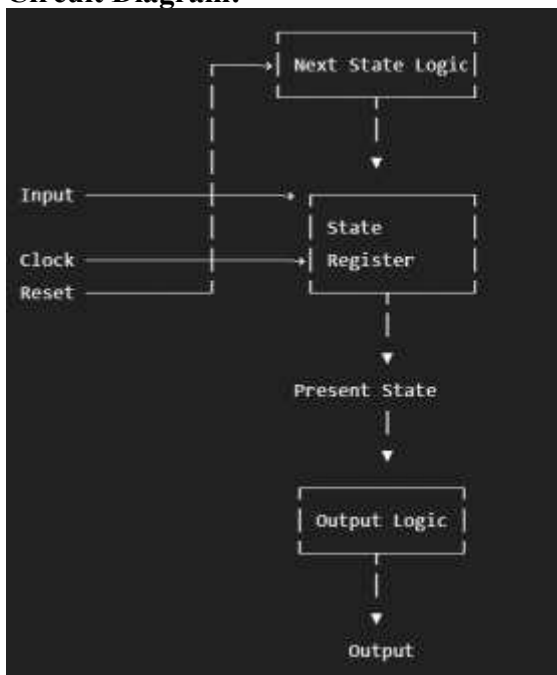
$$Present\ State = S2$$

and

$$Input = 1$$

The output becomes HIGH when the FSM is in state S2 and the current input is 1, indicating detection of the sequence 101.

Circuit Diagram:



Verilog Code:

Design Module :

```

module sequence_detector_101
  ( input wire clk,
    input wire reset,
    input wire in,
    output reg detected
  );

  reg [1:0] state, next_state;

  parameter S0 = 2'b00,

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

S1 = 2'b01,
S2 = 2'b10;

```
// State register
always @(posedge clk) begin
    if (reset)
        state <= S0;
    else
        state <= next_state;
end
```

```
// Next-state and output logic
always @(*) begin
    next_state = S0;
    detected = 1'b0;
```

```
case (state)
```

```
    S0: begin
        if (in)
            next_state = S1;
        else
            next_state = S0;
    end
```

```
    S1: begin
        if (in)
            next_state = S1;
        else
            next_state = S2;
    end
```

```
    S2: begin
        if (in) begin
            next_state = S1;
            detected = 1'b1;
        end
        else
            next_state = S0;
    end
```

```
    default: begin
        next_state = S0;
        detected = 1'b0;
    end
```

```
endcase
end
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

endmodule

TESTBENCH :

```
module sequence_detector_101_tb;
```

```
reg clk;
reg reset;
reg in;
```

```
wire detected;
```

```
sequence_detector_101 uut (
    .clk(clk),
    .reset(reset),
    .in(in),
    .detected(detected)
);
```

```
// Clock generation
always #5 clk = ~clk;
```

```
initial begin
```

```
    clk = 0;
    reset = 1;
    in = 0;
```

```
    #10;
    reset = 0;
```

```
    // Input sequence: 1 0 1
    #10 in = 1;
    #10 in = 0;
    #10 in = 1;
```

```
    // Additional inputs
    #10 in = 1;
    #10 in = 0;
    #10 in = 1;
```

```
    #20;
```

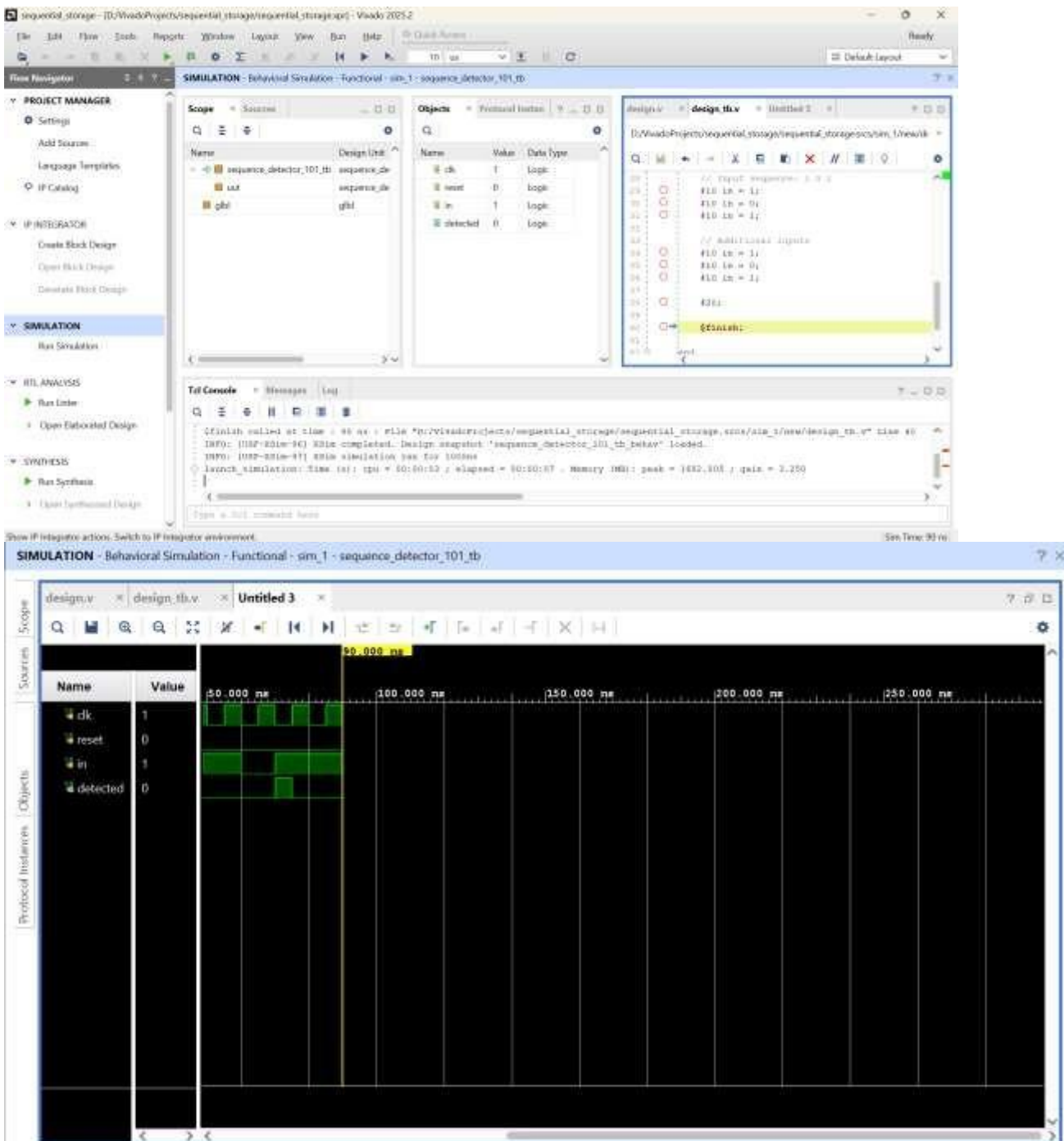
```
    $finish;
```

```
end
```

```
endmodule
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

Simulation Results:



The simulation waveform verifies the state transitions of the FSM. When the input sequence 101 is applied, the FSM transitions through the required states and the output detected becomes HIGH when the complete sequence is recognized. The reset signal initializes the FSM to the initial state. Thus, the expected FSM behavior is verified through simulation.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

Conclusion:

The FSM-based controller was successfully designed and implemented using Verilog HDL. The FSM was simulated using Vivado Simulator, and the state transitions and output behavior were verified for the applied input sequence. The RTL schematic and simulation waveform confirmed the correct operation of the FSM-based sequence detector.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733025

Post Lab Exercise:

1. Why is a Mealy FSM generally faster than a Moore FSM?

A Mealy FSM produces output based on both the present state and input. Therefore, the output can respond to an input change without waiting for an additional state transition.

2. How does allowing overlapping sequences affect the number of FSM states?

Allowing overlapping sequences can reduce the number of states because a portion of a detected sequence can also serve as the beginning of the next sequence.

3. Why is state minimization important in FSM design?

State minimization reduces the number of states and therefore reduces the required hardware resources and complexity of the FSM.

4. What is the impact of an incorrect state transition in an FSM?

An incorrect state transition can cause the FSM to enter an unintended state, resulting in incorrect outputs and improper controller operation.

5. Why is state encoding important in digital system implementation?

State encoding determines how FSM states are represented using binary values. A suitable encoding can improve area, speed, and implementation efficiency.

6. How does one-hot encoding improve FPGA-based FSM performance?

In one-hot encoding, each state is represented by one dedicated flip-flop. This can simplify the combinational logic required for state transitions and may improve FSM speed on FPGAs.

7. How does an FSM improve the reliability of a digital controller?

An FSM provides clearly defined states and transitions, making the controller behavior predictable, structured, and easier to verify.

8. Why are reset states essential in FSM design?

A reset state initializes the FSM to a known starting condition. This prevents the FSM from starting in an undefined state and ensures predictable operation.